
FAST NUCES Karachi

Deep Learning and Perception

Reconstructed Final Examination Paper with Model Solutions

University:	FAST NUCES Karachi
Teacher:	Jawwad Shamsi
Course Name:	Deep Learning and Perception
Course Code:	CS-4045
Date:	22 May 2026
Total Marks / Time:	100 Marks / 3 Hours

Note: This document is reconstructed from memory for revision purposes. The questions have 90–95% similarity, but the scenario and exact numerical values may be different (but they are still solvable and their answers are provided too for verification). If the actual question paper is found then ignore this paper and solution and use the original question paper for learning (use help of AI to solve it and give solutions if there is need).

Prepared from student memory; not an official paper.

Q1. Feed Forward and Backpropagation in a Neural Network

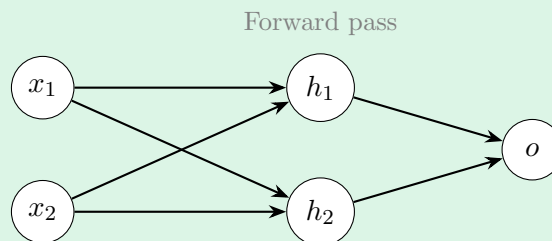
Consider a neural network with 2 input neurons, one hidden layer with 2 neurons, and one output neuron. Each hidden neuron has its own bias. The following values are given:

Quantity	Given Values
Inputs	$x_1 = 0.6, \quad x_2 = 0.8$
Input to hidden weights	$w_{x_1,h_1} = 0.2, \quad w_{x_2,h_1} = 0.4, \quad w_{x_1,h_2} = 0.5, \quad w_{x_2,h_2} = 0.3$
Hidden biases	$b_{h_1} = 0.1, \quad b_{h_2} = 0.2$
Hidden to output weights	$w_{h_1,o} = 0.7, \quad w_{h_2,o} = 0.6$
Output bias	$b_o = 0.3$
Target output	$t = 1$
Learning rate	$\eta = 0.1$

- Perform forward propagation and calculate the output of hidden neurons h_1 , h_2 , and the final output neuron o .
- Calculate the error/loss between the predicted output and target output.
- Perform backpropagation and update the weights from the hidden layer to the output layer.

Answer to Question 1

One Hidden Layer Neural Network



Backpropagation: output error flows backward to update weights

a) Forward propagation

$$\text{net}_{h_1} = (0.6)(0.2) + (0.8)(0.4) + 0.1 = 0.54$$

$$h_1 = \sigma(0.54) = 0.6318$$

$$\text{net}_{h_2} = (0.6)(0.5) + (0.8)(0.3) + 0.2 = 0.74$$

$$h_2 = \sigma(0.74) = 0.6770$$

$$\text{net}_o = (0.6318)(0.7) + (0.6770)(0.6) + 0.3 = 1.1485$$

$$o = \sigma(1.1485) = \mathbf{0.7592}$$

Therefore, the network prediction is approximately **0.7592**.

b) Loss calculation

$$E = \frac{1}{2}(t - o)^2 = \frac{1}{2}(1 - 0.7592)^2 = \mathbf{0.0290}$$

c) Backpropagation for hidden-to-output weights

$$\begin{aligned}\delta_o &= (t - o) \cdot o(1 - o) \\ &= (1 - 0.7592)(0.7592)(1 - 0.7592) = 0.0440\end{aligned}$$

$$w_{h_1,o}^{\text{new}} = 0.7 + (0.1)(0.0440)(0.6318) = \mathbf{0.7028}$$

$$w_{h_2,o}^{\text{new}} = 0.6 + (0.1)(0.0440)(0.6770) = \mathbf{0.6030}$$

Final updated weights: $w_{h_1,o} = 0.7028$ and $w_{h_2,o} = 0.6030$.

Q2. CNN Feature Map Size and Parameter Calculation

An RGB image of size $19 \times 19 \times 3$ is given as input to a CNN. The network applies the following layers:

- **First convolution layer:** 3×3 filters, 16 filters, stride 1, padding 0
- **Second convolution layer:** 3×3 filters, 32 filters, stride 1, padding 0
- **Pooling layer:** 2×2 pooling window with stride 2

- Calculate the output dimensions after each convolution and pooling layer.
- Calculate the number of learnable parameters in both convolution layers.
- Explain why CNNs are more parameter-efficient than fully connected neural networks for image classification tasks.

Answer to Question 2

a) & b) Output dimensions and parameter calculations

Layer	Input	Operation	Output Dim.	Params
Input	—	RGB image	$19 \times 19 \times 3$	0
Conv 1	$19 \times 19 \times 3$	3×3 , 16 filters, S=1, P=0	$17 \times 17 \times 16$	$(3 \times 3 \times 3) \times 16 + 16 = 448$
Conv 2	$17 \times 17 \times 16$	3×3 , 32 filters, S=1, P=0	$15 \times 15 \times 32$	$(3 \times 3 \times 16) \times 32 + 32 = 4,640$
Pooling	$15 \times 15 \times 32$	2×2 pool, stride 2	$7 \times 7 \times 32$	0
Flatten	$7 \times 7 \times 32$	Convert to vector	1,568	0

Note: The pooling output is $7 \times 7 \times 32$ because a 2×2 window with stride 2 does not exactly divide 15; the usual calculation floors the spatial result.

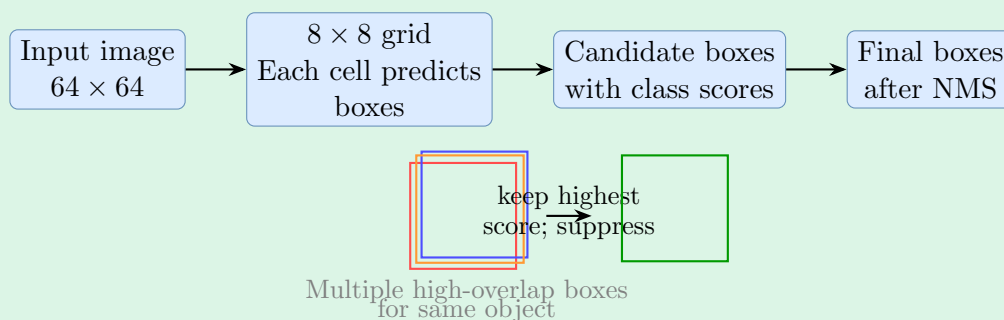
c) Parameter efficiency explanation

- CNNs use *local connectivity*: each filter looks at a small region instead of connecting every input pixel to every neuron.
- CNNs use *parameter sharing*: the same filter weights are reused across the whole image.
- A fully connected neural network would need separate weights for many pixel-to-neuron connections, which becomes very large for images.
- CNNs also preserve spatial structure, so they learn edges, curves, textures, and shapes more naturally than a flattened dense network.

Q3. Object Detection Using YOLO

A hospital wants to build a YOLO-based object detection system. The system receives an input image of size 64×64 and divides it into an 8×8 grid. Each grid cell predicts 2 bounding boxes. The system should detect the following 5 classes: Ambulance, Patient, Doctor, Stretcher, Wheelchair.

- Identify the number of classes and the number of bounding boxes predicted per grid cell.
- Determine the dimension of the YOLO output for the given image.
- In YOLO, multiple bounding boxes may be predicted for the same object. Explain why this happens. What is the method used to remove duplicate boxes? Explain the method.

Answer to Question 3**YOLO Output and Non-Max Suppression****a) Number of classes and bounding boxes**

$$C = 5 \quad (\text{classes}) \quad \text{Bounding boxes per grid cell} = 2$$

b) YOLO output dimension

Per bounding box: x, y, w, h , confidence = 5 values

Per grid cell: $2 \times (5 + 5) = 20$ values

Grid size: 8×8

Total output: $8 \times 8 \times 20 = 1,280$ values

c) Duplicate boxes and Non-Max Suppression (NMS)

YOLO can predict multiple boxes for the same object because nearby grid cells or multiple box predictors may detect the same object with different confidence scores. The method used to remove duplicate boxes is called **Non-Max Suppression (NMS)**.

NMS works by:

1. Selecting the bounding box with the highest confidence score.
2. Calculating how much it overlaps with other boxes using *Intersection over Union* (IoU).
3. Suppressing / removing boxes that overlap too much with the selected box.

This leaves only the best bounding box for each object.

Q4. Variational Autoencoders and GANs

A bank wants to build a model that can generate and restore handwritten numbers on scanned cheques. Sometimes the scanned cheque numbers become distorted, unclear, or partially damaged. The model should help generate or reconstruct clear digit images.

- a) Explain how both a Variational Autoencoder and a Generative Adversarial Network can be used for this task. For each model, explain its architecture, training process, and generation/restoration process.
- b) Draw a general Variational Autoencoder architecture diagram. Label the diagram and write the relevant equations showing how a VAE learns.
- c) Compare VAE and GAN in terms of image quality, diversity/variety of generated images, and ease of training.

Answer to Question 4**a) VAE and GAN approach****Variational Autoencoder approach:**

- A VAE uses an *encoder* to compress a cheque digit image into a latent representation and a *decoder* to reconstruct a clean digit image from that representation.
- During training, the input can be a distorted digit and the target can be the clean or original digit. The model learns to reconstruct the correct digit while

keeping the latent space smooth.

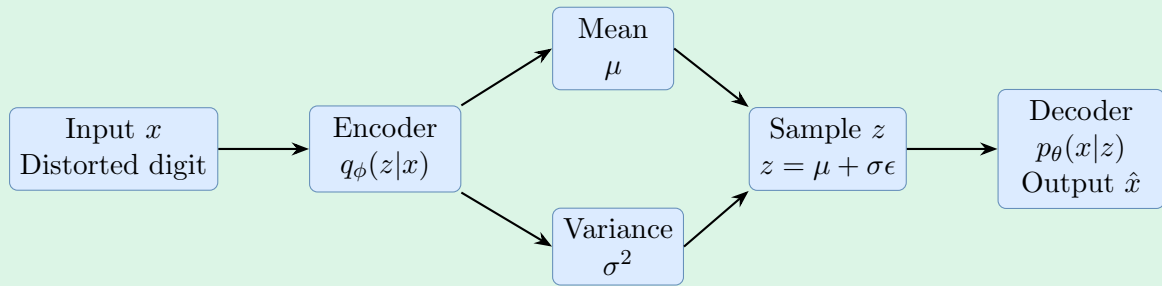
- For generation, a random latent vector can be sampled and passed through the decoder to produce a digit-like image.

GAN approach:

- A GAN uses a *generator* and a *discriminator*. The generator tries to create realistic clean digit images, while the discriminator tries to distinguish real cheque digits from generated ones.
- During training, the generator improves by learning to fool the discriminator. The discriminator improves by learning to detect fake images.
- For restoration, a conditional GAN can be used where the generator takes a damaged digit image as input and outputs a clearer restored digit image.

b) General VAE diagram and equations

Variational Autoencoder (VAE)



Training balances reconstruction accuracy with a smooth latent distribution.

Key equations:

$$\begin{aligned}
 q_{\phi}(z|x) &= \mathcal{N}(\mu, \sigma^2) \\
 z &= \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \\
 \hat{x} &= p_{\theta}(x|z) \\
 \mathcal{L} &= \|x - \hat{x}\|^2 + D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z)) \\
 D_{\text{KL}} &= -\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)
 \end{aligned}$$

The reconstruction loss teaches the model to rebuild the input clearly. The KL divergence keeps the latent space close to a normal distribution, which makes sampling and generation possible.

c) VAE vs. GAN comparison

Aspect	VAE	GAN
Image quality	Usually smoother or slightly blurry because it optimises reconstruction and distribution regularisation.	Often sharper and more realistic because the discriminator pushes the generator toward realistic samples.
Diversity / variety	Usually good diversity because the latent space is continuous and structured.	Can generate high-quality variety, but may suffer from mode collapse where it produces limited types of outputs.
Ease of training	Generally easier and more stable to train.	Harder and less stable because two networks compete during training.

Q5. Sequence Modelling Using RNN, Bi-RNN, and LSTM

For each of the following scenarios, identify whether RNN, Bi-RNN, or LSTM would be most suitable. Justify your answer and draw the relevant architecture diagram with important equations.

- Predicting a missing word in the middle of a sentence. Example: *“The student studied very _____ because he wanted to pass the exam.”*
- Predicting the next day’s stock value using previous stock data and news content as input.
- Classifying the sentiment of tweets as positive, negative, or neutral.

Answer to Question 5

a) Missing word in the middle of a sentence

The suitable model is a **Bi-RNN** because the blank depends on both previous words and future words. Words before the blank such as “studied very” and words after the blank such as “because he wanted to pass the exam” both help predict the missing word, likely “hard”.

$$\begin{aligned}
 h_t^f &= f(W_{xh} x_t + W_{hh} h_{t-1}^f + b) \\
 h_t^b &= f(W_{xh} x_t + W_{hh} h_{t+1}^b + b) \\
 h_t &= [h_t^f ; h_t^b]
 \end{aligned}$$

Sequence Modelling Choices



Bi-RNN: uses left and right context for middle blank

b) Stock value prediction using previous data and news

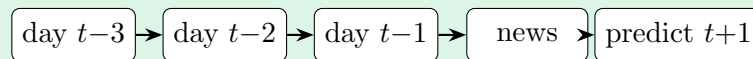
The suitable model is an **LSTM** because stock prediction may depend on long-term trends, older events, and sequential dependencies. LSTM can remember useful past information and forget irrelevant information through gates.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$



LSTM: keeps useful long-term sequential information.

c) Tweet sentiment classification

The suitable model is an **LSTM** because tweets, despite being short, can contain negations, sarcasm, and context-dependent sentiment that create long-range dependencies within the sequence. LSTM's gating mechanism lets it remember relevant context and forget irrelevant tokens, making it better at capturing the true sentiment than a vanilla RNN which suffers from vanishing gradients even over short sequences. The task is still *many-to-one*: the full tweet sequence is read and one final sentiment label is produced.

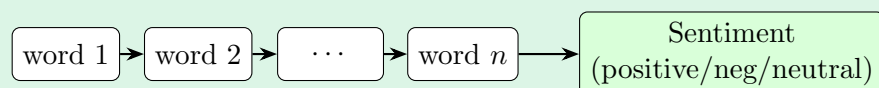
$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{forget gate})$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{input gate})$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{cell state update})$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{output gate})$$

$$y = \text{softmax}(W_y h_T + b_y) \quad (\text{final sentiment class})$$



LSTM many-to-one: full tweet processed, final hidden state classifies sentiment.

Q6. Transformers and Attention Mechanism

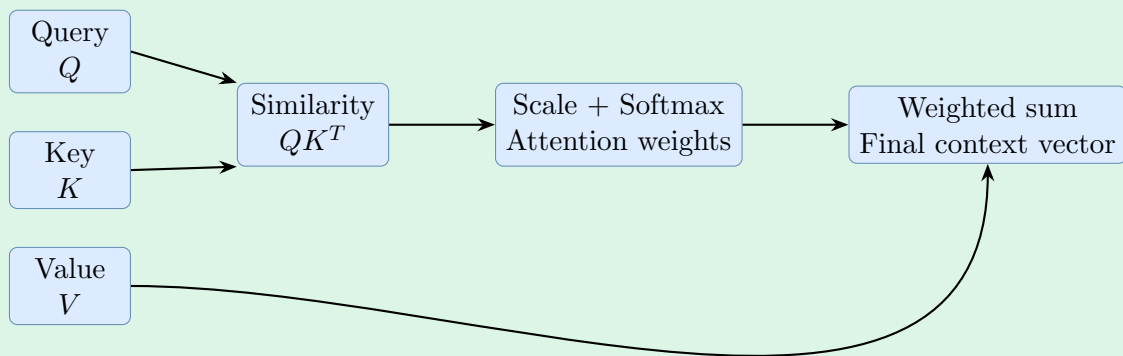
- a) Describe the attention mechanism used in Transformers. Include a diagram and the main equation.

- b) What is positional encoding? Explain why it is required in Transformers and what role it plays in understanding word order.
- c) Explain the three types of Transformer architectures with diagrams. Also state which Transformer architecture would be most suitable for building a chatbot. Justify your answer and briefly explain how the chatbot generates a response when a prompt is given.

Answer to Question 6

a) Attention mechanism

Scaled Dot-Product Attention



Idea: choose which tokens matter most, then mix their value vectors.

Attention allows the model to focus on the most relevant tokens while processing a sequence. Each token is converted into Query, Key, and Value vectors. The Query is compared with Keys to calculate attention scores. These scores are normalised and used to take a weighted sum of the Values.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

The scaling factor $\sqrt{d_k}$ prevents the dot products from becoming too large, which helps softmax remain stable.

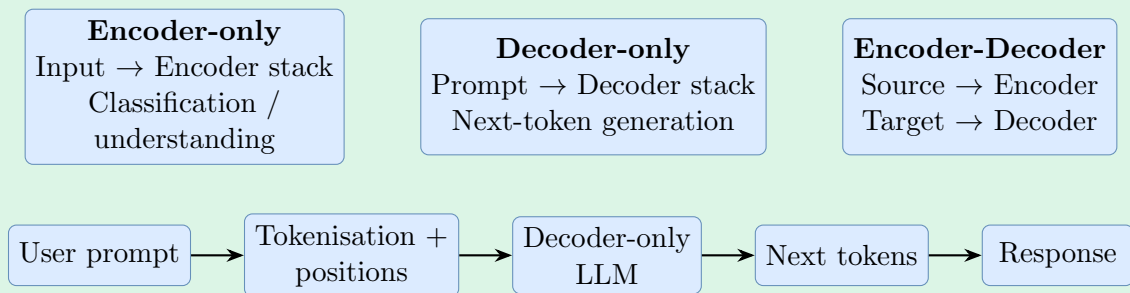
b) Positional encoding

Transformers process tokens in parallel, so they do not naturally know the order of words. Positional encoding adds position information to token embeddings so the model can understand sequence order.

For example, “Dog bites man” and “Man bites dog” contain the same words but have different meanings because their word order is different. Positional encoding helps the Transformer capture this difference.

c) Transformer types and chatbot architecture

Transformer Architecture Types



For a chatbot, decoder-only is commonly used because it generates text autoregressively.

Type	Main idea	Example use
Encoder-only	Uses encoder blocks to understand the full input context.	Classification, embeddings, sentiment analysis
Decoder-only	Uses masked self-attention to generate tokens one by one.	Chatbots and text generation
Encoder-Decoder	Encoder reads input; decoder generates output based on encoder representation.	Translation and summarisation

For a chatbot, a **decoder-only** Transformer is most suitable because it generates a response autoregressively. The prompt is tokenised, positional information is added, masked self-attention processes the prompt, and the model predicts the next token repeatedly until the final response is produced.

Q7. Reinforcement Learning in a Warehouse Robot

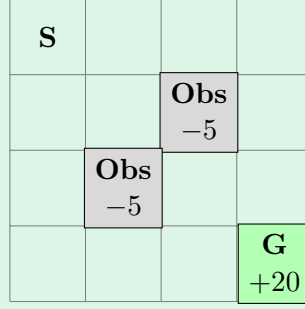
A robot moves inside a 4×4 warehouse grid. It can move left, right, up, and down.

- Reaching the goal gives a reward of +20.
- Each normal step gives a reward of -1.
- If the robot encounters an obstacle, it gets a reward of -5.

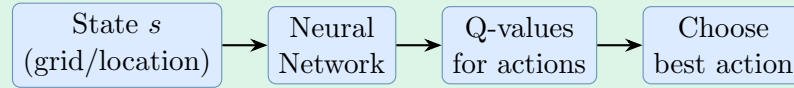
- What is the state space in this scenario?
- What is Q-learning? Write its equation and explain each term.
- What is the biggest limitation of Q-based learning? How do Deep Q-Networks solve this problem? Illustrate your answer.

Answer to Question 7

Warehouse Robot: Q-Learning and Deep Q-Network



State = robot cell. Actions = up, down, left, right.



DQN replaces the huge Q-table with a neural network approximator.

a) State space

The state space is the set of all possible positions of the robot in the 4×4 grid. Since there are 16 cells, the maximum number of states is 16. If obstacle cells are not valid positions, then the usable state space is smaller.

$$S = \{(\text{row}, \text{column}) \text{ positions in the } 4 \times 4 \text{ grid}\}$$

$$A = \{\text{up, down, left, right}\}$$

b) Q-learning

Q-learning is a value-based reinforcement learning algorithm. It learns the quality of taking an action in a particular state. The learned Q-value tells the agent how good an action is expected to be.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- $Q(s, a)$: current value of taking action a in state s
- α : learning rate
- r : reward received after taking the action
- γ : discount factor for future reward
- s' : next state
- $\max_{a'} Q(s', a')$: best estimated future value from the next state

c) Limitation of Q-based learning and DQN solution

The biggest limitation of tabular Q-learning is *scalability*. For large or continuous environments, the Q-table becomes too large because it must store a value for every state-action pair.

Deep Q-Networks (DQN) solve this problem by replacing the Q-table with a neural network. The input is the state, and the output is a Q-value for each possible

action. This allows the model to generalise to many states instead of memorising every state-action value separately.

Q8. Short Questions

Answer the following short questions.

- i) What are MCP and A2A? Explain their purpose in modern AI/agent systems.
- ii) Explain why a normal fully connected neural network is not practical for sentiment classification when the input is a sentence. Illustrate your answer with a diagram.
- iii) Explain the effect of learning rate in neural network training. Illustrate the difference between a very low learning rate and a very high learning rate.
- iv) Why is it important to compare training accuracy and validation accuracy during model training? Explain using a diagram or illustration.
- v) What is hallucination in Large Language Models? Explain one or more possible solutions and illustrate the idea.
- vi) Explain the Siamese Network architecture used for face verification/recognition. Draw the architecture and explain how it is trained.
- vii) Explain triplet loss in the context of face recognition.
- viii) Draw and explain a deepfake system that swaps a person's face in a video and produces a swapped-face output video.
- ix) How is a Vision-Language Model trained? Explain and illustrate it.

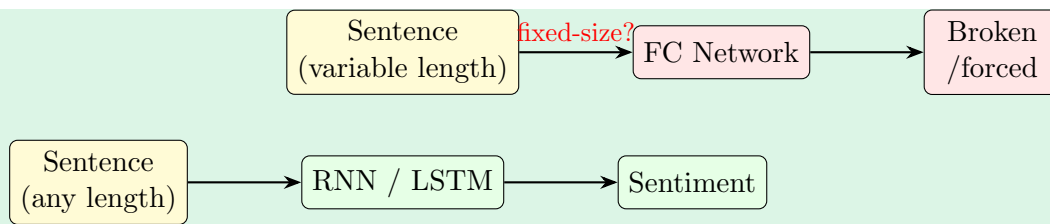
Answer to Question 8

i) MCP and A2A

MCP (Model Context Protocol) is used to connect AI systems with external tools, data sources, files, APIs, and context in a standardised way. **A2A** (Agent-to-Agent communication) allows different AI agents to communicate, coordinate, and delegate tasks to each other.

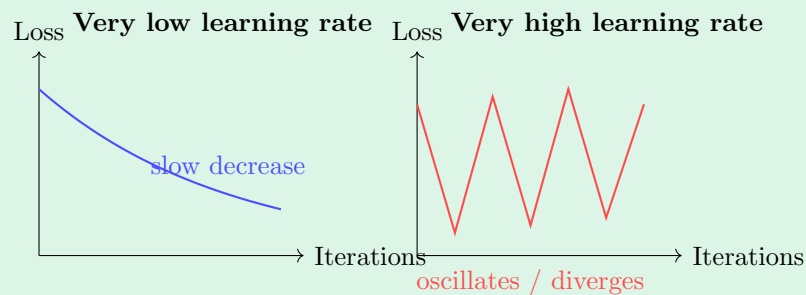
ii) Why a normal fully connected NN is not practical for sentence sentiment classification

A sentence is sequential data. Word order matters, sentence length can vary, and the meaning of a word may depend on nearby words. A normal fully connected neural network expects a fixed-size input and does not naturally remember word order or context. Therefore, RNNs, LSTMs, or Transformers are more suitable.



iii) Learning rate effect

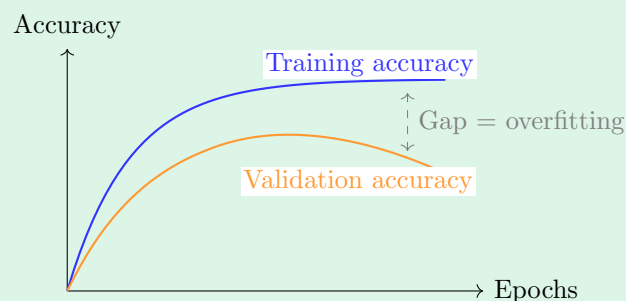
Learning Rate Effects



A very low learning rate makes training extremely slow because weights update by tiny amounts. A very high learning rate can overshoot the minimum and cause unstable training or divergence. A good learning rate helps the loss decrease steadily.

iv) Training accuracy vs. validation accuracy

Training vs. Validation Accuracy

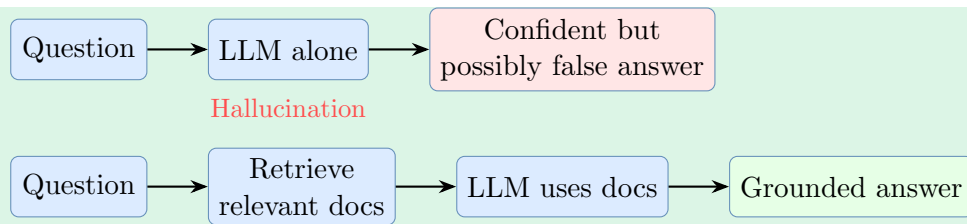


Training accuracy shows how well the model performs on the data it learned from. Validation accuracy shows how well it performs on unseen data. If training accuracy is high but validation accuracy is low, the model is *overfitting*. If both are low, the model is *underfitting*.

v) Hallucination and solution

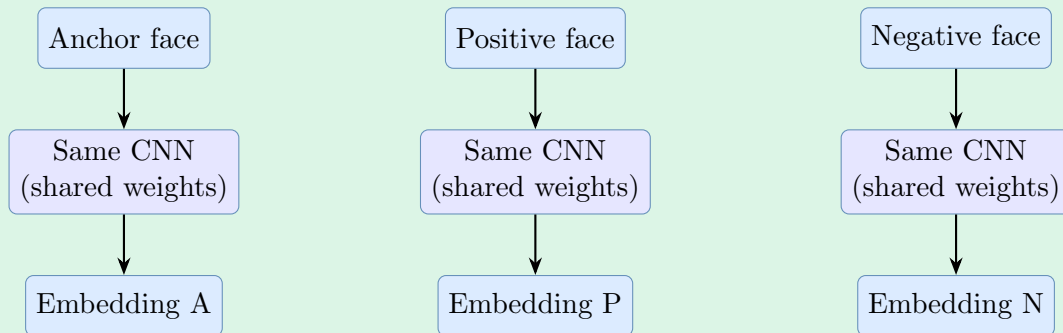
Hallucination means an LLM gives an answer that sounds confident but is factually wrong or unsupported. Solutions include RAG, better prompting, tool use, citations, validation checks, and grounding the model in reliable documents.

Hallucination and RAG-style Grounding



vi) Siamese Network

Siamese Network and Triplet Loss for Face Recognition



Training goal: $d(A, P)$ small, $d(A, N)$ large.

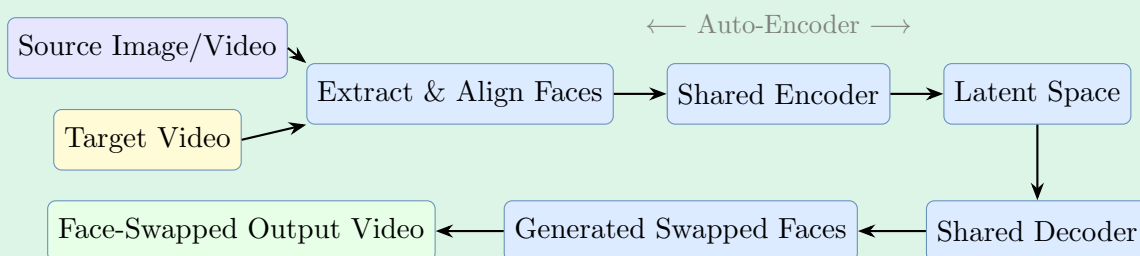
A Siamese Network uses two or more identical neural networks with shared weights. Each face image is passed through the same network to produce an embedding. The model is trained so that embeddings of the same person are close together and embeddings of different people are far apart.

vii) Triplet loss

Triplet loss uses three images: an *anchor* image, a *positive* image of the same person, and a *negative* image of a different person. The objective is to make the anchor-positive distance smaller than the anchor-negative distance by at least a margin.

$$\mathcal{L} = \max(d(A, P) - d(A, N) + \text{margin}, 0)$$

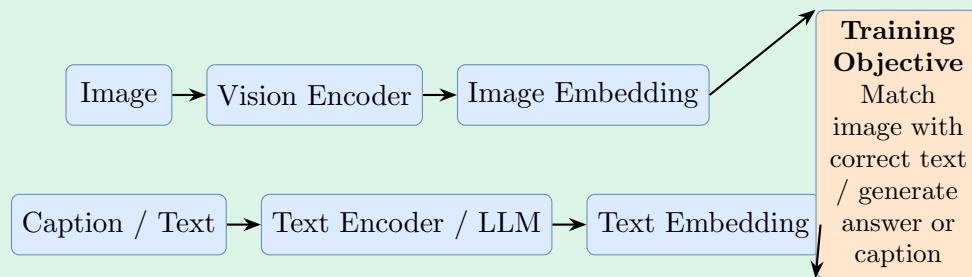
viii) Deepfake face swap



A deepfake face swap system takes a source face and a target video. It detects and aligns faces in the target video, generates swapped face frames, blends them into the target frames, and outputs a video where the target person's face appears replaced by the source identity.

ix) VLM training

Vision-Language Model Training



A Vision-Language Model is trained using paired image-text data. The image is processed by a vision encoder, and text is processed by a text encoder or language model. Training can involve matching images with correct captions, generating captions, answering questions about images, or aligning visual and textual embeddings in the same semantic space.